UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 – Redes

Sección 20

Jorge Yass



Proyecto 2 – Fase 2

Ana Laura Tschen 221645

Fernando Rueda 23748

Luis Felipe Aguilar Portillo 23195

Fernando Hernández 23645

Esteban Cárcamo 23016

Guatemala, 29 de octubre 2026

Organización del equipo

El trabajo se dividió por áreas para que cada integrante se encargara de una parte del diseño y la implementación de la red.

Integrante	Carné	Rol	Aportes
Ana Laura Tschen	221645	Diseño y documentación	Diagrama de red y redacción de la propuesta técnica.
Fernando Hernández	23645	Infraestructura de red	Configuración inicial de Terraform, VPC, subredes CIDR/VLSM, Internet Gateway y tablas de rutas.
Luis Felipe Aguilar Portillo	23195	Seguridad y cómputo	Security Groups por segmento, instancias EC2 y outputs de Terraform.
Fernando Rueda	23748	Despliegue y pruebas	Documentación del despliegue, implementación en AWS, pruebas de conectividad y capturas.
Esteban Cárcamo	23016	Segmentación	Cálculos de subneteo CIDR/VLSM (Excel) y README del repositorio.

Tabla 1. Roles y aportes de cada integrante

Nombre de la empresa

El nombre Barrilete Ventures representa la misión de impulsar a startups guatemaltecas para que alcancen nuevas oportunidades de crecimiento. El barrilete simboliza creatividad, visión y progreso, además de mantener una conexión con la identidad cultural de Guatemala. La palabra Ventures comunica su relación con el emprendimiento, la innovación y la inversión en empresas emergentes. En conjunto, el nombre refleja una organización que brinda capital, orientación y apoyo estratégico para ayudar a nuevos proyectos a llegar más lejos.

Giro del negocio

Para efectos de este proyecto, Barrilete Ventures se modela como una empresa guatemalteca dedicada a la inversión ángel y al capital de riesgo. Su actividad consiste en identificar startups en etapas tempranas, evaluar oportunidades de inversión y brindar capital, mentoría y acompañamiento estratégico para apoyar su crecimiento en Guatemala y América Latina.

Nombre de dominio

El dominio seleccionado es barrilete.dev. La extensión .dev refuerza la identidad digital de la empresa y su relación con el desarrollo, la innovación y el ecosistema tecnológico. Además, proporciona un nombre breve y fácil de asociar con la plataforma web.

Relación del negocio con la red propuesta

- Ventas (25 usuarios): contacto con emprendedores e inversionistas, seguimiento de oportunidades y gestión comercial.
- TI (15 usuarios): soporte de la plataforma, administración de usuarios, seguridad y operación de los servicios en la nube.
- Data Center (5 servidores): capacidad prevista para el sitio web, aplicación, base de datos, documentos y monitoreo.
- Visitas (50 usuarios): acceso aislado para emprendedores, inversionistas y proveedores, sin comunicación directa con las redes privadas.

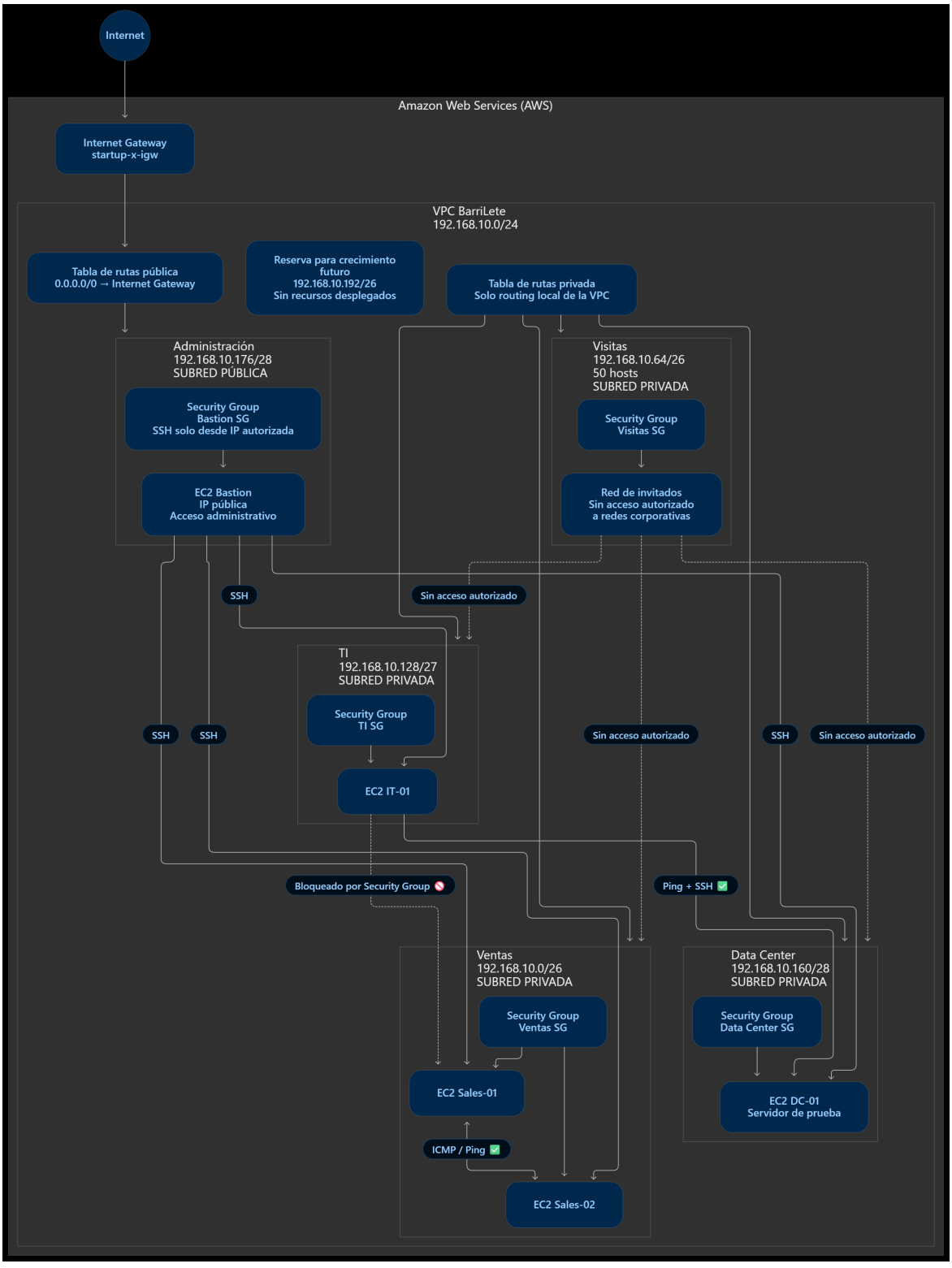
- Recursos Humanos (agregado en la Fase 2): administra la intranet de RRHH, la contratación y la planilla. Sus equipos tienen su propia subred y sus usuarios entran por SSH con su cuenta del directorio LDAP.

Justificación del Proveedor Elegido

Utilizamos Amazon Web Services (AWS) como proveedor de nube. La red se implementó con Amazon Virtual Private Cloud (Amazon VPC), donde se creó el bloque 192.168.10.0/24 con las subredes de Ventas, Visitas, TI, Data Center y Administración, y con Amazon EC2 se crearon las instancias para las pruebas de conectividad.

Se eligió AWS por tres razones principales. Primero, el equipo contaba con créditos de uso en AWS, por lo que se decidió aprovecharlos para implementar la red sin incurrir en costos adicionales ni abrir cuentas en otro proveedor. Segundo, Amazon VPC permite dividir la red en subredes con su propio bloque CIDR, tablas de rutas y Security Groups, lo que facilita separar Ventas, TI, Data Center y Visitas, y decidir qué tráfico puede pasar entre ellas. En otros proveedores, como Digital Ocean, cada VPC es un único rango privado sin subredes internas, por lo que cada departamento tendría que ser una VPC separada; en AWS todos los segmentos viven dentro de una sola VPC y el tráfico entre ellos se controla con Security Groups. Tercero, AWS tiene soporte completo en Terraform, lo que permitió definir toda la infraestructura como código, crearla y destruirla cuando fuera necesario, y así no gastar los créditos más rápido de lo necesario. Además, Amazon Route 53 permitirá administrar el dominio barrilete.dev cuando se publique la página web en las siguientes fases.

Diagrama de red



Cálculos de segmentación de la red

Para la implementación del proyecto se utilizará segmentación CIDR/VLSM. Esta alternativa permite asignar a cada departamento una subred de tamaño acorde con sus necesidades, en lugar de asignar bloques del mismo tamaño a todas las áreas. La segmentación tradicional mediante cuatro redes /26 es funcional, pero desperdicia una cantidad considerable de direcciones, especialmente en el Data Center, que actualmente únicamente requiere cinco servidores.

La red base de la VPC es 192.168.10.0/24, con 256 direcciones (de 192.168.10.0 a 192.168.10.255). Antes de calcular el tamaño de cada subred se tomaron en cuenta dos restricciones de Amazon VPC:

a) AWS reserva 5 direcciones en cada subred, no 2: la dirección de red, la .1 (router de la VPC), la .2 (servidor DNS), la .3 (reservada para uso futuro) y la de broadcast. Por eso, una subred con h bits de host tiene $2^h - 5$ IP utilizables.

b) AWS solo permite subredes con prefijos entre /16 y /28. Una subred /29, como la que bastaría para el Data Center, no se puede crear.

Paso 1. Tamaño de cada subred

Se ordenaron los segmentos de mayor a menor necesidad y, para cada uno, se buscó el menor número de bits de host (h) que cumpliera $2^h - 5 \geq$ hosts requeridos, respetando el mínimo /28 de AWS. El prefijo se obtiene como /32 − h.

Segmento	Hosts requeridos	Cálculo	Bits de host (h)	Prefijo	Máscara
Visitas	50	$2^5 - 5 = 27$ no alcanza; $2^6 - 5 = 59 \geq 50$	6	/26	255.255.255.192
Ventas	25	$2^5 - 5 = 27 \geq 25$, pero solo deja 2 IP libres; se usa $2^6 - 5 = 59$	6	/26	255.255.255.192
TI	15	$2^4 - 5 = 11$ no alcanza; $2^5 - 5 = 27 \geq 15$	5	/27	255.255.255.224
Data Center	5	Con /29 bastaría ($2^3 - 2 = 6$), pero el mínimo de AWS es /28: $2^4 - 5 = 11 \geq 5$	4	/28	255.255.255.240
Administración	1 (Bastion)	Mínimo de AWS: $2^4 - 5 = 11 \geq 1$	4	/28	255.255.255.240
Reserva	—	Espacio restante del /24: $2^6 = 64$ direcciones	6	/26	255.255.255.192

Tabla 2. Tamaño de cada subred

Paso 2. Asignación de los bloques

Las subredes se asignaron de forma contigua a partir de 192.168.10.0, de la más grande a la más pequeña, para que cada bloque inicie en un múltiplo de su tamaño. El tamaño del bloque es 256 menos el último octeto de la máscara: 256 − 192 = 64 para /26, 256 − 224 = 32 para /27 y 256 − 240 = 16 para /28. Ventas y Visitas tienen el mismo tamaño (/26), por lo que el orden entre ellas no afecta la alineación; se colocó Ventas primero.

Así, Ventas ocupa de .0 a .63, Visitas de .64 a .127, TI de .128 a .159, Data Center de .160 a .175, Administración de .176 a .191 y la reserva de .192 a .255. Cada subred inicia en la dirección siguiente al broadcast de la anterior, por lo que ninguna se traslapa con otra.

Paso 3. Cálculo binario de la dirección de red y de broadcast

La dirección de red se obtiene con la operación AND entre una dirección IP de la subred y la máscara, y la de broadcast con la operación OR entre la dirección de red y el inverso de la máscara; el valor entre paréntesis es el decimal del octeto. Como todas las máscaras tienen 255 (11111111) en los tres primeros octetos, estos se mantienen en 192.168.10 y el cálculo solo cambia en el cuarto octeto. En la tabla se toma como ejemplo la IP de una instancia de cada subred. El desarrollo completo de los 32 bits está en el archivo de Excel de segmentación (hoja CIDR).

Segmento	IP de ejemplo	IP (4.º octeto)	Máscara (4.º octeto)	AND = red	Inverso de la máscara	OR = broadcast
Ventas	192.168.10.10 (Sales-01)	00001010 (10)	11000000 (192)	00000000 (0)	00111111 (63)	00111111 (63)
Visitas	192.168.10.74 (Guest-01, opcional)	01001010 (74)	11000000 (192)	01000000 (64)	00111111 (63)	01111111 (127)
TI	192.168.10.138 (IT-01)	10001010 (138)	11100000 (224)	10000000 (128)	00011111 (31)	10011111 (159)
Data Center	192.168.10.164 (DC-01)	10100100 (164)	11110000 (240)	10100000 (160)	00001111 (15)	10101111 (175)
Administración	192.168.10.180 (Bastion)	10110100 (180)	11110000 (240)	10110000 (176)	00001111 (15)	10111111 (191)
Reserva	192.168.10.192 (sin instancias)	11000000 (192)	11000000 (192)	11000000 (192)	00111111 (63)	11111111 (255)

Tabla 3. Cálculo binario del cuarto octeto

Resultado de la segmentación

Segmento	Dirección de red	Máscara	Broadcast	Rango utilizable en AWS	IP válidas ($2^h - 2$)	IP utilizables en AWS ($2^h - 5$)	Hosts requeridos
Ventas	192.168.10.0/26	255.255.255.192	192.168.10.63	192.168.10.4 – 192.168.10.62	62	59	25
Visitas	192.168.10.64/26	255.255.255.192	192.168.10.127	192.168.10.68 – 192.168.10.126	62	59	50
TI	192.168.10.128/27	255.255.255.224	192.168.10.159	192.168.10.132 – 192.168.10.158	30	27	15
Data Center	192.168.10.160/28	255.255.255.240	192.168.10.175	192.168.10.164 – 192.168.10.174	14	11	5
Administración	192.168.10.176/28	255.255.255.240	192.168.10.191	192.168.10.180 – 192.168.10.190	14	11	1 (Bastion)
Reserva	192.168.10.192/26	255.255.255.192	192.168.10.255	192.168.10.196 – 192.168.10.254	62	59	Crecimiento futuro

Tabla 4. Segmentación final de la red 192.168.10.0/24

Las cinco subredes que se crean en AWS ocupan 192 de las 256 direcciones (75 %), y el bloque 192.168.10.192/26 (64 direcciones, 25 %) queda libre para crecimiento futuro. Las IP privadas de las instancias de prueba están dentro del rango utilizable de su subred: Sales-01 (.10) y Sales-02 (.11) en Ventas, IT-01 (.138) en TI, DC-01 (.164) en Data Center y el Bastion (.180) en

Administración. Con cuatro redes /26 de la segmentación tradicional se ocuparía todo el /24 y no quedaría espacio para la subred de Administración ni para crecer.

Cambios respecto a la segmentación inicial

La primera propuesta usaba la red 192.168.1.0/24 con Visitas /26, Ventas /27, TI /27 y Data Center /29, calculada con la regla clásica de $2^h - 2$ IP válidas. Al implementarla en AWS se ajustó por tres motivos:

a) El Data Center pasó de /29 a /28, porque AWS no permite crear subredes menores a /28.

b) Ventas pasó de /27 a /26: en AWS un /27 deja 27 IP utilizables, apenas 2 más que las 25 personas del equipo, sin margen para crecer.

c) Se agregó la subred de Administración (192.168.10.176/28). No es un departamento: aloja el Bastion, la única instancia con IP pública y el único punto de entrada por SSH hacia las subredes privadas.

Además, se usa como red base 192.168.10.0/24, que es el bloque configurado en la VPC.

Screenshots y explicación de la implementación

Pasos de la implementación

La infraestructura no se creó a mano desde la consola de AWS, sino con Terraform, una herramienta de infraestructura como código. Todos los recursos que aparecen en las capturas están descritos en archivos de configuración (.tf) que se entregan junto con este documento. Así, la red se puede crear y destruir siempre de la misma forma con un par de comandos, lo que nos permitió apagar todo después de cada prueba para no consumir créditos.

Los archivos se organizaron por tipo de recurso:

Archivo	Qué define
versions.tf y provider.tf	Versión de Terraform, proveedor AWS, región us-east-1 y etiquetas comunes de todos los recursos.
variables.tf, locals.tf y terraform.tfvars.example	Los bloques CIDR de cada subred, el tipo de instancia (t3.micro), el nombre de la llave SSH y la IP pública desde la que se permite SSH.
vpc.tf	La VPC barrilete-vpc con el bloque 192.168.10.0/24.
subnets.tf	Las cinco subredes en la zona us-east-1a. Solo la de Administración asigna IP pública.
internet-gateway.tf	El Internet Gateway barrilete-igw.
route-tables.tf	barrilete-public-rt (ruta 0.0.0.0/0 hacia el IGW), asociada solo a Administración, y barrilete-private-rt (solo la ruta local), asociada a Ventas, Visitas, TI y Data Center.
security-groups.tf	Un Security Group por segmento y uno para el Bastion.

ec2.tf	Las instancias Amazon Linux 2023 con IP privada fija.
outputs.tf	Muestra al final las IP de las instancias y el comando SSH para conectarse.

Tabla 5. Archivos de configuración de Terraform

Los pasos que seguimos fueron:

1. Preparación: creamos un usuario IAM con su clave de acceso y la configuramos en el equipo (~/.aws/credentials). En EC2 creamos el par de llaves barrillete-key (tipo ED25519) para entrar por SSH. Luego copiamos terraform.tfvars.example a terraform.tfvars y colocamos nuestra IP pública con máscara /32.
2. terraform init: descarga el proveedor de AWS y prepara la carpeta de trabajo.
3. terraform plan: muestra todo lo que se va a crear sin modificar nada en AWS. Revisamos que fueran 25 recursos.
4. terraform apply: crea los recursos respetando sus dependencias: primero la VPC, después las subredes y el Internet Gateway, luego las tablas de rutas con sus asociaciones, los Security Groups y por último las instancias. El proceso toma alrededor de tres minutos.
5. Verificación: revisamos en la consola de AWS que todo se creara correctamente (capturas de esta sección) y realizamos las pruebas de conectividad.
6. terraform destroy: al terminar eliminamos todos los recursos para no generar costos.

El control del tráfico entre segmentos se hace con los Security Groups. Todas las subredes pueden enrutarse entre sí por la ruta local de la VPC, pero un Security Group solo deja pasar lo que tiene una regla explícita y bloquea todo lo demás:

Security Group	Tráfico de entrada permitido
barrilete-bastion-sg	SSH (puerto 22) solo desde la IP pública del administrador.
barrilete-sales-sg	SSH desde el Bastion; ICMP (ping) solo entre instancias de Ventas.
barrilete-it-sg	SSH desde el Bastion.
barrilete-datacenter-sg	SSH desde el Bastion y desde TI; ICMP solo desde TI.
barrilete-guest-sg	Ninguno.

Tabla 6. Reglas de entrada de los Security Groups

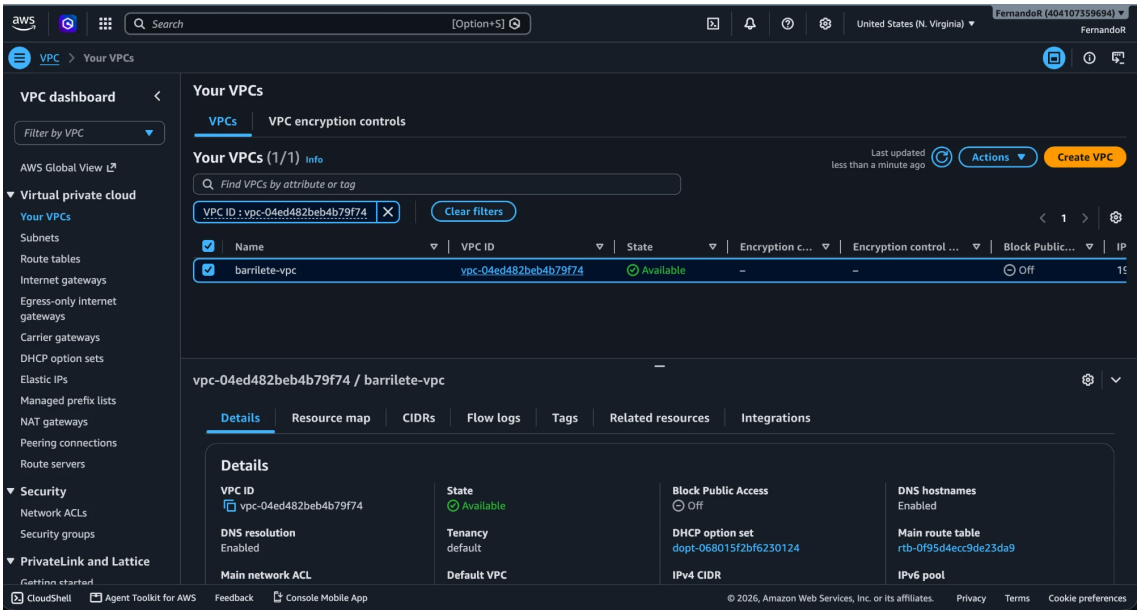
Acceso a las instancias privadas

Ventas, Visitas, TI y Data Center son subredes privadas: sus instancias no tienen IP pública ni ruta hacia Internet. Para entrar a ellas usamos el Bastion como punto de salto. Primero cargamos la llave con ssh-add barrillete-key.pem y después nos conectamos con ssh -J ec2-user@<IP pública del Bastion> ec2-user@<IP privada de destino>. Para entrar desde TI al Data Center agregamos la opción -A, que reenvía la llave sin copiarla al servidor.

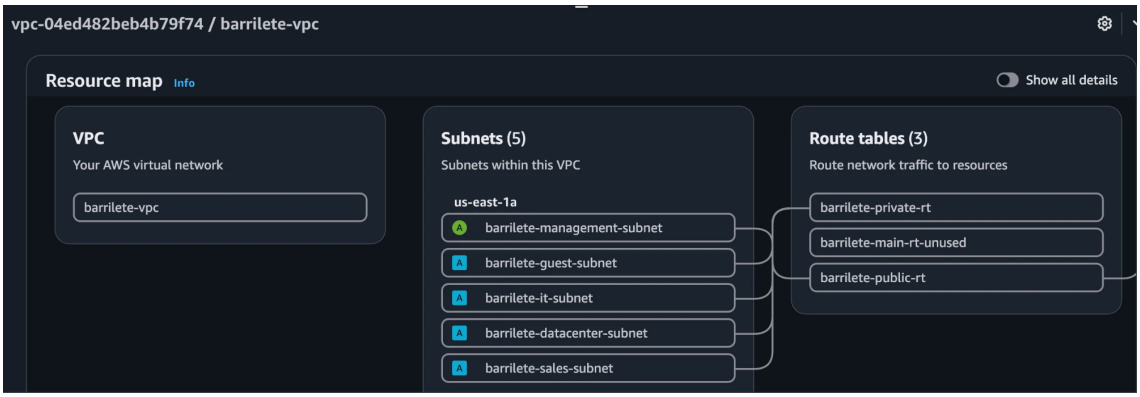
Las IP privadas de las instancias están fijas en Terraform (Sales-01 192.168.10.10, Sales-02 192.168.10.11, IT-01 192.168.10.138, DC-01 192.168.10.164 y Bastion 192.168.10.180), por lo que son las mismas cada vez que se despliega la red. La IP pública del Bastion sí cambia en cada

despliegue: en las pruebas aparece 3.235.87.197 y en la captura de instancias 3.228.3.81, porque el ambiente se volvió a crear entre ambas tomas.

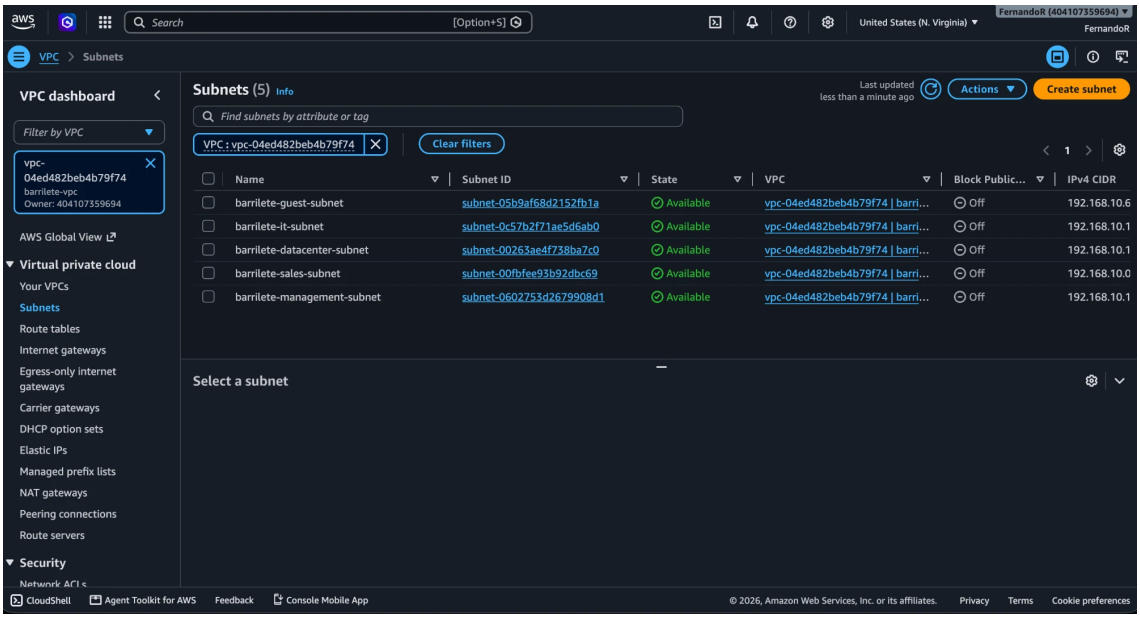
Esta es la VPC barrilete-vpc, ya creada y en estado Available. Es la red base donde vive todo el proyecto, con el bloque 192.168.10.0/24.



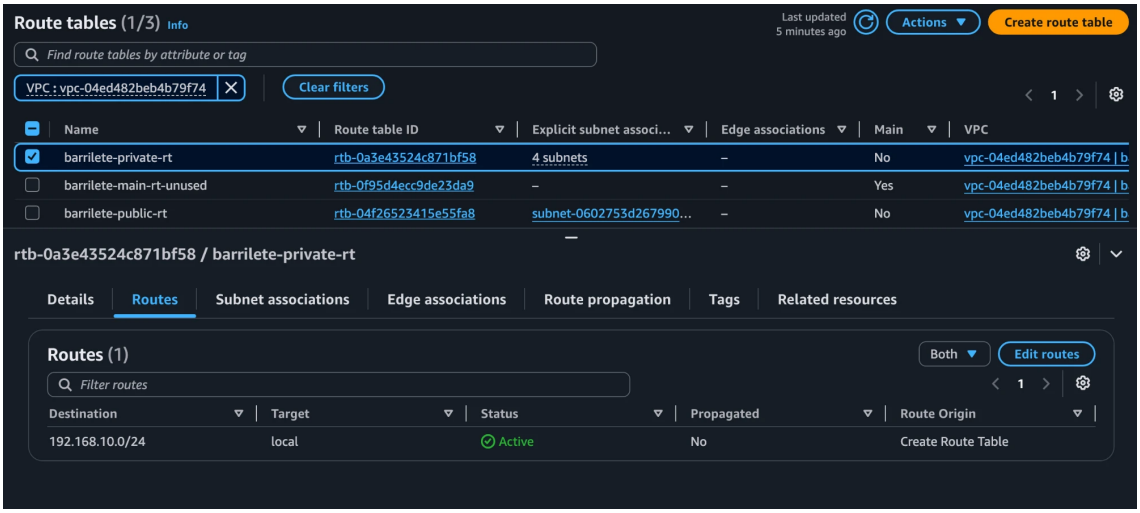
En el resource map se ve cómo queda todo conectado: las 5 subredes (management, guest, it, datacenter y sales) en us-east-1a y las 3 tablas de rutas a las que están asociadas.



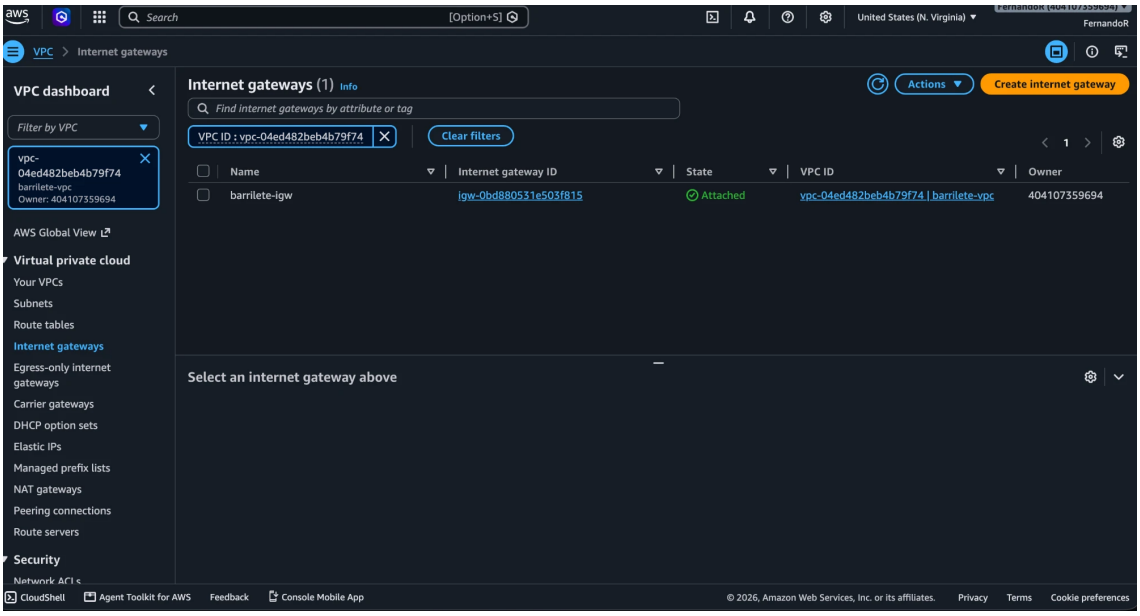
Aquí están las 5 subredes dentro de 192.168.10.0/24, todas en Available. Cada una corresponde a un área de la red.



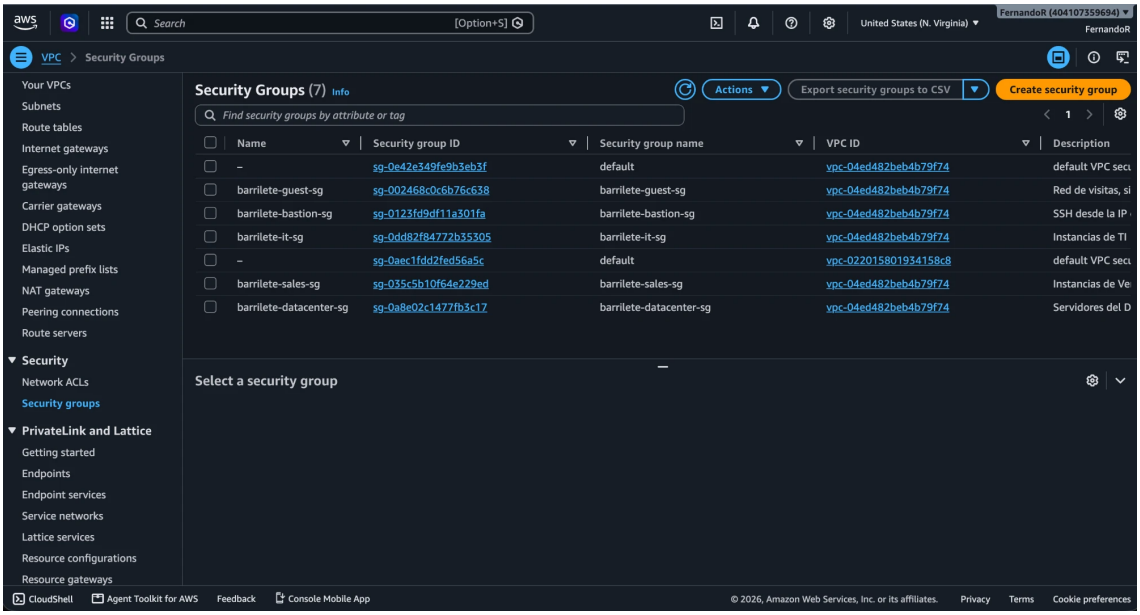
La tabla barrilete-private-rt está asociada a las 4 subredes privadas y solo tiene la ruta local 192.168.10.0/24, o sea que esas subredes no tienen salida a internet. La barrilete-public-rt solo va con la subred de management y la main se queda sin usar.



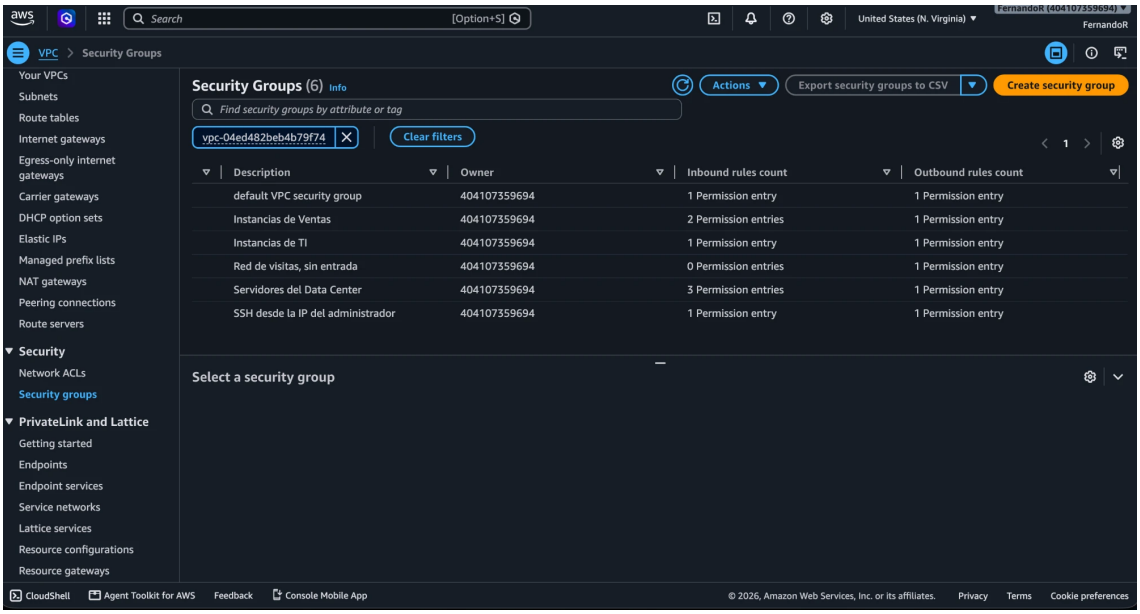
El Internet Gateway barrilete-igw está Attached a la VPC. Es la única salida a internet y solo la usa la subred pública.



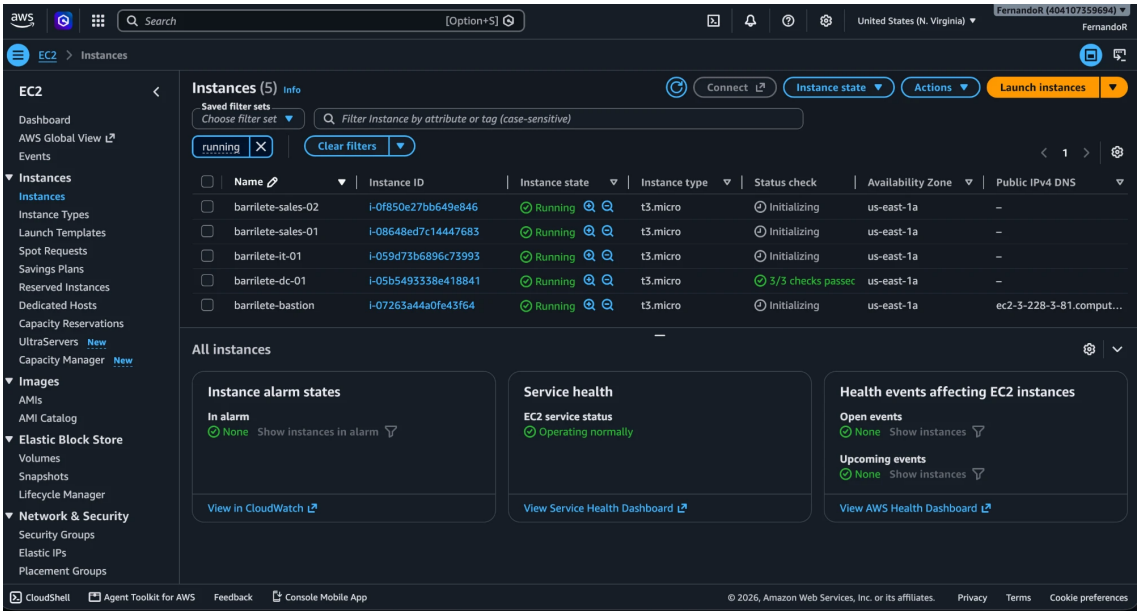
Estos son los security groups de la VPC: uno por área (guest, it, sales y datacenter) más el del bastion.



Acá se ve cuántas reglas tiene cada uno: el del data center tiene 3 de entrada, ventas 2, TI 1, el bastion 1 (SSH solo desde la IP del administrador) y visitas ninguna, así que nadie le puede entrar.

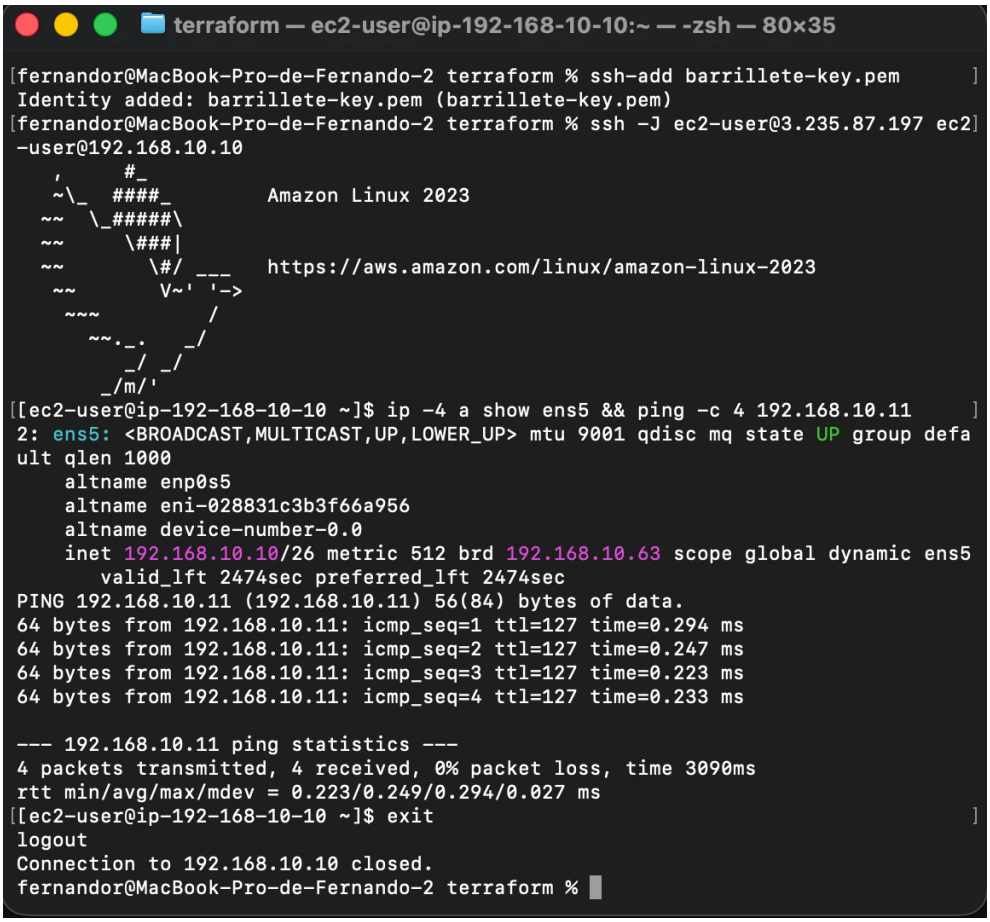


Por último, las 5 instancias t3.micro corriendo: sales-01, sales-02, it-01, dc-01 y el bastion, que es el único con IP pública.

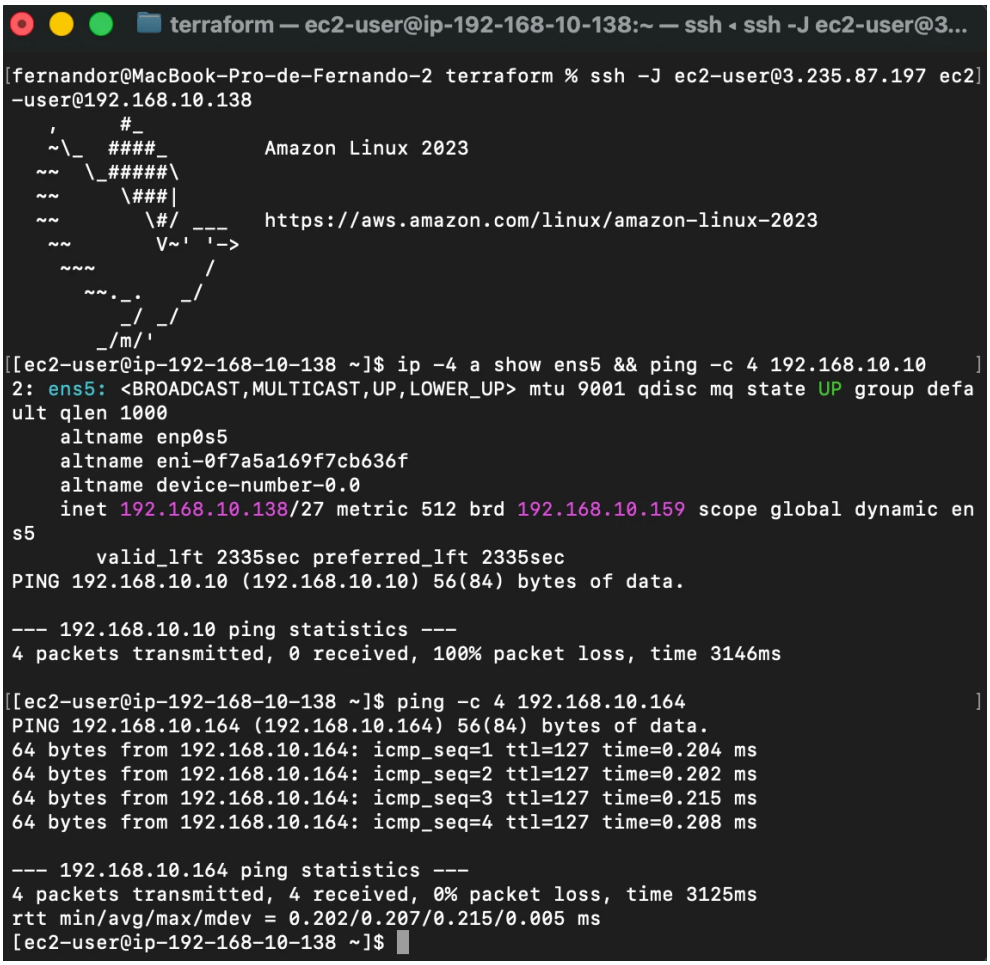


Ejemplos de pruebas de conectividad

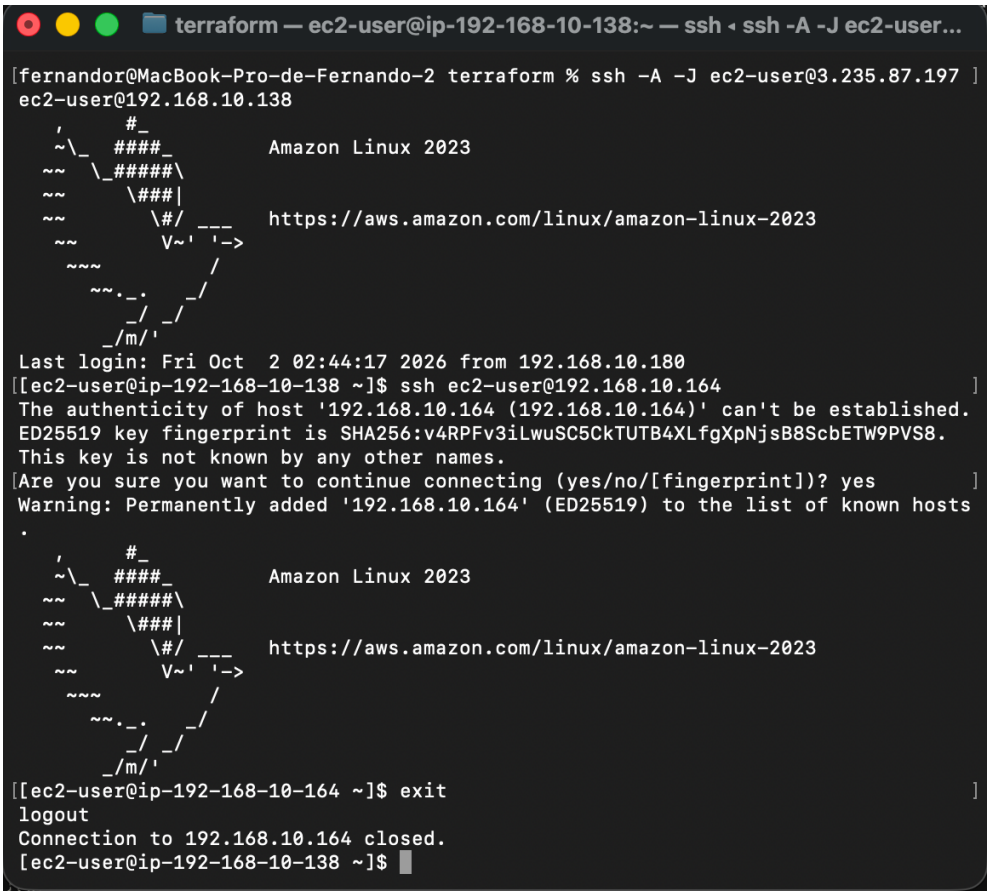
Prueba 1 (misma subred): entramos a sales-01 (192.168.10.10) pasando por el bastion y le hacemos ping a sales-02 (192.168.10.11), que está en la misma subred de ventas. Responde sin pérdida, así que dentro de ventas sí hay comunicación.



Pruebas 2 y 3 (entre subredes): desde it-01 (192.168.10.138) el ping a sales-01 (192.168.10.10) se pierde al 100%, porque ventas no acepta tráfico de TI. En cambio el ping a dc-01 (192.168.10.164) sí responde, porque el data center sí deja entrar a TI.



Prueba 4 (administración por SSH): desde it-01 entramos por SSH a dc-01 (192.168.10.164) sin problema, usando el bastion como salto y reenviando la llave con -A.


terraform — ec2-user@ip-192-168-10-138:~ — ssh ◂ ssh -A -J ec2-user...
[fernandor@MacBook-Pro-de-Fernando-2 terraform % ssh -A -J ec2-user@3.235.87.197]
ec2-user@192.168.10.138
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
Last login: Fri Oct 2 02:44:17 2026 from 192.168.10.180
[[ec2-user@ip-192-168-10-138 ~]$ ssh ec2-user@192.168.10.164]
The authenticity of host '192.168.10.164 (192.168.10.164)' can't be established.
ED25519 key fingerprint is SHA256:v4RPFv3iLwuSC5CkTUTB4XLfgXpNjsB8ScbETW9PVS8.
This key is not known by any other names.
[Are you sure you want to continue connecting (yes/no/[fingerprint])? yes]
Warning: Permanently added '192.168.10.164' (ED25519) to the list of known hosts
.
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
[[ec2-user@ip-192-168-10-164 ~]$ exit]
logout
Connection to 192.168.10.164 closed.
[ec2-user@ip-192-168-10-138 ~]$

Fase 2: Implementación de los servicios

En esta fase continuamos la infraestructura de la Fase 1 en AWS y le agregamos los servicios que pide el enunciado: HTTP para la intranet de Recursos Humanos, DNS para llegar a ella por nombre, LDAP para administrar y autenticar a los usuarios, un firewall con NAT, listas de control de acceso y hardening de las instancias. Igual que en la Fase 1, todo está descrito en Terraform, de modo que un solo terraform apply crea la red, levanta los servicios y los deja configurados sin pasos manuales.

Organización del equipo en la Fase 2

Integrante	Carné	Rol	Aportes en la Fase 2
Fernando Hernández	23645	Red	Subred de RRHH, firewall con NAT (nftables), tabla de rutas privada hacia el firewall y NACL de Visitas.
Fernando Rueda	23748	LDAP y despliegue	CA y certificados generados con Terraform, servidor OpenLDAP, clientes SSSD, despliegue en AWS, pruebas y capturas.
Luis Felipe Aguilar Portillo	23195	Seguridad y cómputo	Security Groups de TI, RRHH y la web, instancias hr-01 y guest-01 y hardening de las reglas.
Esteban Cárcamo	23016	Segmentación, DNS y web	Segmentación con la subred de RRHH, zona DNS privada, Nginx de la intranet y README.
Ana Laura Tschen	221645	Diseño y documentación	Página Barrilete People, diagrama de la Fase 2, script de pruebas, evidencias y reporte.

Tabla 7. Roles y aportes de la Fase 2

Requerimientos y cómo los cumplimos

Requerimiento	Cómo lo implementamos
HTTP: servidor web interno de RRHH con una página estática	Nginx en dc-01 (Data Center) con la página Barrilete People.
DNS: acceder a la intranet por nombre y no por IP	Zona privada de Route 53 barrilete.dev con rrhh.barrilete.dev. Nginx cierra las peticiones que llegan por IP.
LDAP: administración y autenticación de usuarios	OpenLDAP en dc-ldap (Data Center) con LDAPS en el puerto 636.
SSH de Ventas y RRHH solo si están autorizados en LDAP	SSSD en sales-01, sales-02 y hr-01. Cada instancia solo acepta a su grupo LDAP y a ti-admins.
Toda la administración de usuarios en LDAP	Los usuarios y grupos solo existen en el directorio. Las instancias no tienen cuentas locales para ellos.
LDAP y la intranet en el Data Center	dc-01 (.164) y dc-ldap (.170) están en 192.168.10.160/28.
Visitas con Internet pero sin acceso a las demás subredes	Salida por el firewall con NAT. La NACL de Visitas bloquea el resto de la VPC.
TI llega a todas las subredes y ninguna llega a TI	Todos los Security Groups aceptan tráfico desde el de TI. El de TI no acepta nada de los departamentos.
Intranet accesible desde todas las subredes menos Visitas, y no desde Internet	El Security Group de la web solo permite el puerto 80 desde Ventas, RRHH, TI, Data Center y Administración. dc-01 no tiene IP pública.

Requerimiento	Cómo lo implementamos
Firewall (puede incluir NAT) y ACL	Instancia barrilete-fw con nftables, NAT y política drop. NACL en la subred de Visitas.
Hardening	Security Groups con lo mínimo, solo los servicios necesarios escuchando, sin login de root y sin LDAP sin cifrar.

Tabla 8. Requerimientos de la Fase 2 y su implementación

Cambios en la segmentación: subred de RRHH

El enunciado pide que los usuarios de Recursos Humanos entren por SSH a sus instancias, así que RRHH necesitaba su propia subred. La tomamos de la reserva que dejamos en la Fase 1 (192.168.10.192/26), sin mover ninguna de las cinco subredes originales. Usamos un /28 porque es el bloque más pequeño que AWS permite y deja 11 IP utilizables, suficientes para los equipos de prueba del área.

El cálculo binario del cuarto octeto es igual al de la Tabla 3. Para la dirección de red hacemos 196 AND 240, o sea 11000100 AND 11110000 = 11000000 (192), y para el broadcast hacemos 192 OR 00001111 = 11001111 (207). Con esto la reserva queda de 192.168.10.208 a 192.168.10.255 (48 direcciones), que no se crea en AWS.

Segmento	Dirección de red	Máscara	Broadcast	Rango utilizable en AWS	IP válidas ($2^h - 2$)	IP utilizables en AWS ($2^h - 5$)
RRHH	192.168.10.192/28	255.255.255.240	192.168.10.207	192.168.10.196 – 192.168.10.206	14	11
Reserva	192.168.10.208 – .255	—	—	—	—	48 direcciones libres

Tabla 9. Subred agregada en la Fase 2

Diagrama de red actualizado

El diagrama muestra la red con los componentes nuevos: la subred de RRHH, el firewall con NAT en Administración, los servidores web y LDAP en el Data Center, la zona DNS privada y los flujos permitidos y bloqueados entre subredes.

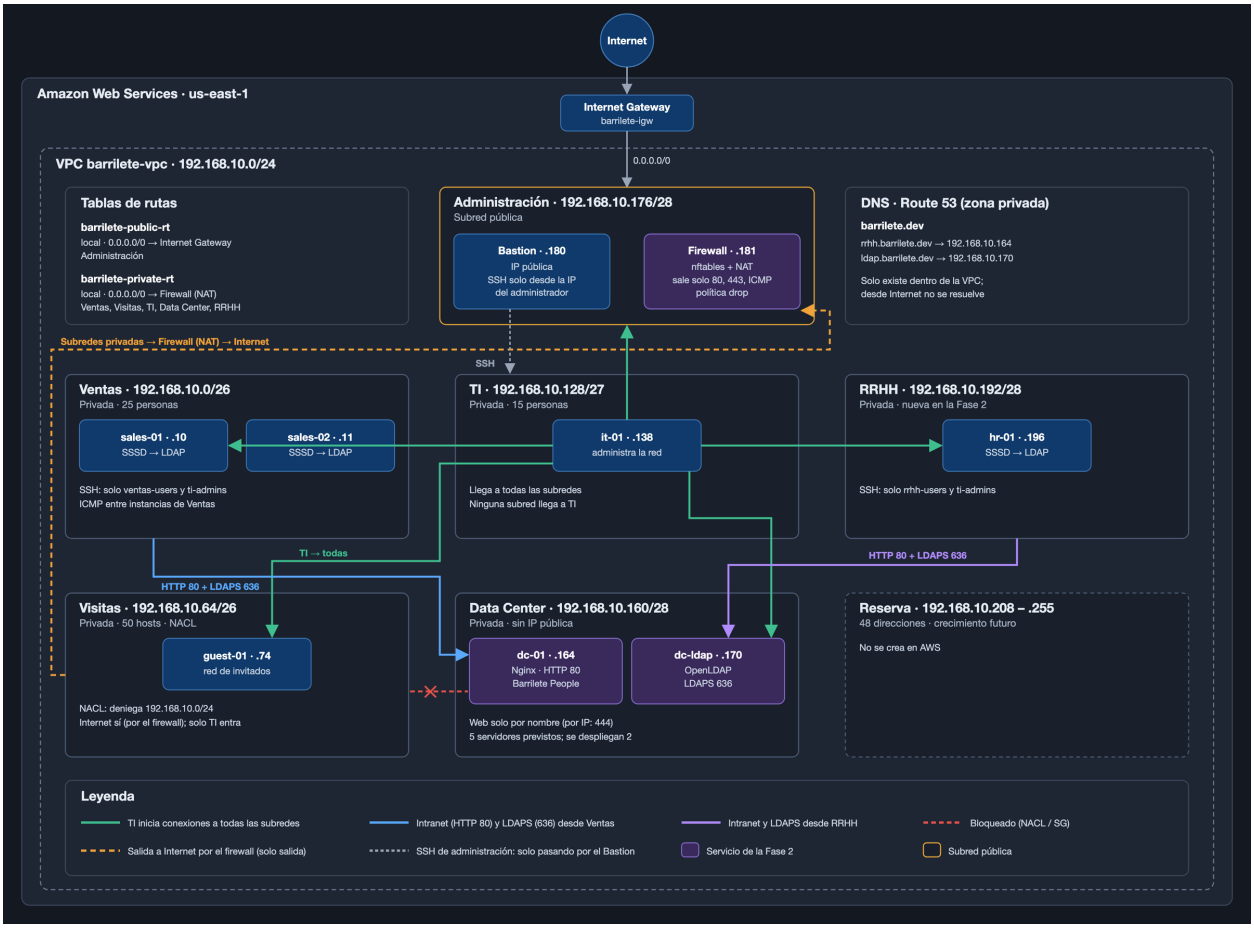


Figura 1. Diagrama de red de la Fase 2

Instancias y direccionamiento

Las IP privadas siguen fijas en Terraform, así que son las mismas en cada despliegue. Cambiamos el tipo de instancia de t3.micro a t2.micro porque la cuenta tiene un límite de 16 vCPU: con las instancias nuevas t3.micro (2 vCPU cada una) nos pasábamos del límite, mientras que t2.micro usa 1 vCPU.

Instancia	Subred	IP privada	Función
barrilete-bastion	Administración	192.168.10.180	Punto de entrada por SSH desde la IP del administrador. Tiene IP pública.
barrilete-fw	Administración	192.168.10.181	Firewall y NAT con nftables. Tiene IP pública para la salida a Internet.
barrilete-sales-01 / -02	Ventas	192.168.10.10 / .11	Equipos de Ventas con login LDAP (SSSD).

Instancia	Subred	IP privada	Función
barrilete-it-01	TI	192.168.10.138	Administración de la red. Llega a todas las subredes.
barrilete-dc-01	Data Center	192.168.10.164	Servidor web de la intranet (Nginx).
barrilete-dc-ldap	Data Center	192.168.10.170	Servidor de directorio (OpenLDAP con LDAPS).
barrilete-hr-01	RRHH	192.168.10.196	Equipo de RRHH con login LDAP (SSSD).
barrilete-guest-01	Visitas	192.168.10.74	Equipo de prueba de la red de visitas.

Tabla 10. Instancias de la Fase 2

Firewall y NAT

En lugar de un NAT Gateway de AWS usamos una instancia propia, barrilete-fw, con Amazon Linux 2023 y nftables. Así cumplimos con el firewall con funciones de NAT que pide el enunciado, podemos mostrar y controlar sus reglas, y cuesta bastante menos que un NAT Gateway. Para que la instancia pueda reenviar tráfico que no es suyo desactivamos el source/destination check y activamos ip_forward.

La tabla de rutas privada, que comparten Ventas, Visitas, TI, Data Center y RRHH, ahora tiene la ruta 0.0.0.0/0 hacia la interfaz del firewall. En nftables la cadena forward tiene política drop y solo deja salir hacia Internet HTTP (80), HTTPS (443) y ping, que es lo que las instancias necesitan para instalar paquetes. Lo que se descarta queda registrado con el prefijo FW-DROP. La cadena postrouting hace masquerade, es decir, cambia la IP privada de origen por la del firewall. La cadena input también es drop y solo acepta SSH desde Administración y TI.

Hay que aclarar que el tráfico dentro de la VPC no pasa por el firewall, porque la ruta local 192.168.10.0/24 siempre tiene prioridad. El firewall controla la salida a Internet, y el aislamiento entre subredes lo hacen los Security Groups y la NACL.

Listas de control de acceso: Security Groups y NACL

Los Security Groups funcionan como un firewall con estado en cada instancia: solo dejan entrar lo que tiene una regla explícita y las respuestas regresan solas. Definimos cada regla como un recurso separado en Terraform para poder agregarlas desde otros archivos. La regla de que TI llega a todas se cumple porque cada Security Group acepta todo el tráfico que viene del grupo de TI, mientras que el de TI solo acepta SSH desde el Bastion (el acceso de administración) y, por el Lab 7, SSH y el puerto 4444 entre las mismas instancias de TI.

Security Group	Tráfico de entrada permitido
barrilete-bastion-sg	SSH (22) solo desde la IP pública del administrador. Todo desde TI.
barrilete-fw-sg	HTTP, HTTPS e ICMP desde la VPC (tráfico que se reenvía a Internet). SSH desde el Bastion. Todo desde TI.
barrilete-sales-sg	SSH desde el Bastion. ICMP entre instancias de Ventas. Todo desde TI.
barrilete-hr-sg	SSH desde el Bastion. ICMP entre instancias de RRHH. Todo desde TI.

Security Group	Tráfico de entrada permitido
barrilete-it-sg	SSH desde el Bastion. SSH y TCP 4444 entre instancias de TI (Lab 7). Nada de los departamentos.
barrilete-web-sg	HTTP (80) desde Ventas, RRHH, Data Center, Bastion y firewall. SSH desde el Bastion. Todo desde TI.
barrilete-ldap-sg	LDAPS (636) solo desde Ventas y RRHH. SSH desde el Bastion. Todo desde TI.
barrilete-guest-sg	Todo desde TI. Nada más.

Tabla 11. Reglas de entrada de los Security Groups en la Fase 2

Además agregamos una Network ACL en la subred de Visitas. A diferencia de los Security Groups, la NACL no tiene estado, se aplica a toda la subred y sí permite reglas de denegación. Las reglas se evalúan en orden de número y la primera que coincide es la que se aplica.

Dirección	Regla	Acción	Origen / destino	Motivo
Salida	100	Permitir	192.168.10.128/27 (TI)	Respuestas a las conexiones que inicia TI
Salida	110	Denegar	192.168.10.0/24	Visitas no llega a ninguna otra subred
Salida	200	Permitir	0.0.0.0/0	Internet, a través del firewall
Entrada	100	Permitir	192.168.10.128/27 (TI)	TI puede entrar a Visitas
Entrada	110	Denegar	192.168.10.0/24	Nada más de la VPC entra
Entrada	200	Permitir	0.0.0.0/0	Respuestas de Internet

Tabla 12. NACL de la subred de Visitas

DNS privado

Para el DNS usamos una zona privada de Amazon Route 53 llamada barrilete.dev, asociada a la VPC. Una zona privada solo responde a las consultas que se hacen desde dentro de la VPC a través del resolver de AWS (192.168.10.2), así que desde Internet el nombre no existe. Tiene dos registros A: rrhh.barrilete.dev apunta a 192.168.10.164 y ldap.barrilete.dev a 192.168.10.170.

Al probar desde cero nos encontramos con que, si una instancia preguntaba por el nombre antes de que la zona terminara de propagarse, el resolver guardaba la respuesta negativa por 15 minutos. Por eso bajamos a 60 segundos el TTL negativo del registro SOA. Visitas también puede resolver el nombre porque está en la misma VPC, pero no se puede conectar, que es lo que pide el enunciado.

Servidor web de la intranet (HTTP)

El servidor web es Nginx en dc-01, en el Data Center, y sirve la página estática Barrilete People. La página y su hoja de estilos están en la carpeta web del repositorio y Terraform las copia a la instancia al arrancar, sin depender de ningún CDN.

Para que la intranet solo funcione por nombre configuramos dos bloques server. El primero es el default_server y responde a cualquier petición con un Host que no conoce (por ejemplo la IP) con el código 444, que en Nginx significa cerrar la conexión sin responder. El segundo solo atiende a server_name rrhh.barrilete.dev. También desactivamos server_tokens para que Nginx no muestre

su versión. La instancia no tiene IP pública y su Security Group no acepta tráfico de Visitas, así que la intranet no es accesible desde Internet ni desde la red de visitas.

LDAP y autenticación por SSH

Instalamos OpenLDAP (slapd) en dc-ldap con Amazon Linux 2023. Configuramos slapd para escuchar solo en ldapi (socket local, para administrarlo) y ldaps en el puerto 636, de modo que en la red nunca viajan contraseñas sin cifrar. Terraform genera una CA propia y el certificado del servidor (con el nombre ldap.barrilete.dev y la IP 192.168.10.170) usando el proveedor tls, y también genera las contraseñas con random_password. Ninguno de estos secretos se sube al repositorio.

El árbol del directorio es dc=barrilete,dc=dev, con ou=People para los usuarios y ou=Groups para los grupos. Para las pruebas creamos un usuario por área. Además, una ACL de OpenLDAP hace que el atributo userPassword solo sirva para autenticarse y nadie lo pueda leer.

Grupo (posixGroup)	GID	Usuario de prueba	UID	Puede entrar por SSH a
ventas-users	10001	vendedor1	20001	sales-01 y sales-02
rrhh-users	10002	rrhh1	20002	hr-01
ti-admins	10003	ti1	20003	Ventas y RRHH

Tabla 13. Usuarios y grupos del directorio LDAP

En sales-01, sales-02 y hr-01 instalamos SSSD como cliente. SSSD consulta los usuarios en ldaps://192.168.10.170, valida el certificado con la CA y autentica la contraseña contra el directorio. La autorización la hace el proveedor simple con simple_allow_groups: en Ventas solo se permiten ventas-users y ti-admins, y en RRHH solo rrhh-users y ti-admins. Así, aunque la contraseña de rrhh1 sea correcta, Ventas le niega el acceso porque su grupo no está autorizado. Con authselect y mkhomedir se crea el directorio personal en el primer login. Toda la administración de usuarios se hace en el LDAP: para dar o quitar acceso basta con cambiar el grupo del usuario en el directorio.

Los clientes usan la IP del servidor LDAP y no su nombre porque, en un despliegue desde cero, SSSD arranca antes de que la zona DNS termine de propagarse. El nombre ldap.barrilete.dev igual queda registrado en el DNS y en el certificado.

Hardening

- Cada Security Group solo abre los puertos del servicio que corre en la instancia. Lo que no tiene regla queda bloqueado.
- Solo el Bastion y el firewall tienen IP pública. El SSH desde Internet solo se acepta desde la IP del administrador.
- Servicios escuchando: SSH en todas las instancias, Nginx en el puerto 80 de dc-01 y slapd en el 636 de dc-ldap. LDAP sin cifrar (389) no está habilitado.

- El firewall tiene política drop en las cadenas input y forward.
- PermitRootLogin no en los clientes LDAP y server_tokens off en Nginx.
- Las contraseñas y llaves privadas viven solo en el estado de Terraform, que no se sube al repositorio.

Archivos de configuración

A los archivos de la Fase 1 (Tabla 5) les agregamos los siguientes. Todos están en el repositorio del proyecto.

Archivo	Qué define
terraform/firewall.tf	Instancia barrilete-fw y su Security Group.
terraform/nacl.tf	NACL de la subred de Visitas.
terraform/dns.tf	Zona privada barrilete.dev, registro SOA y registros rrhh y ldap.
terraform/tls.tf	CA propia y certificado del servidor LDAP.
terraform/ldap.tf	Instancia dc-ldap, su Security Group y las contraseñas de los usuarios.
terraform/subnets.tf, route-tables.tf	Subred de RRHH y ruta 0.0.0.0/0 hacia el firewall en la tabla privada.
terraform/security-groups.tf	Security Groups de la Fase 2 (Tabla 11).
terraform/ec2.tf	Instancias hr-01 y guest-01, y la configuración de arranque de la web y los clientes LDAP.
scripts/fw.sh.tftpl	Configuración de nftables, NAT e ip_forward del firewall.
scripts/ldap.sh.tftpl	Instalación de OpenLDAP, TLS, esquemas, usuarios y grupos.
scripts/sssd-client.sh.tftpl	Cliente SSSD y configuración de SSH en Ventas y RRHH.
scripts/web.sh.tftpl y web/	Nginx de la intranet y la página Barrilete People.
scripts/smoke-tests.sh	Script que corre todas las pruebas de la Fase 2 y muestra el resultado.

Tabla 14. Archivos de configuración agregados en la Fase 2

Pasos de la implementación

El flujo de trabajo fue el mismo de la Fase 1. Corrimos terraform plan para revisar los cambios, terraform apply para crear todo y, al terminar cada sesión, terraform destroy para no generar costos. Cada instancia se configura sola al arrancar con su user_data: el firewall carga sus reglas, dc-ldap instala el directorio, dc-01 levanta Nginx y los clientes esperan a que LDAP responda antes de iniciar SSSD. Después del apply esperamos unos minutos a que terminen esos scripts y corrimos smoke-tests.sh, que prueba todos los requerimientos desde la laptop pasando por el Bastion.

Screenshots y pruebas de la Fase 2

Para entrar a las instancias privadas seguimos usando el Bastion como salto (ssh -J bastion). A guest-01 solo se puede entrar pasando también por it-01, porque la NACL de Visitas solo acepta conexiones de TI.

El terraform apply desde cero creó 86 recursos. En los outputs se ven las IP de todas las instancias, la IP pública del firewall y la URL de la intranet. Las contraseñas de LDAP aparecen como sensitive, así que Terraform no las muestra.

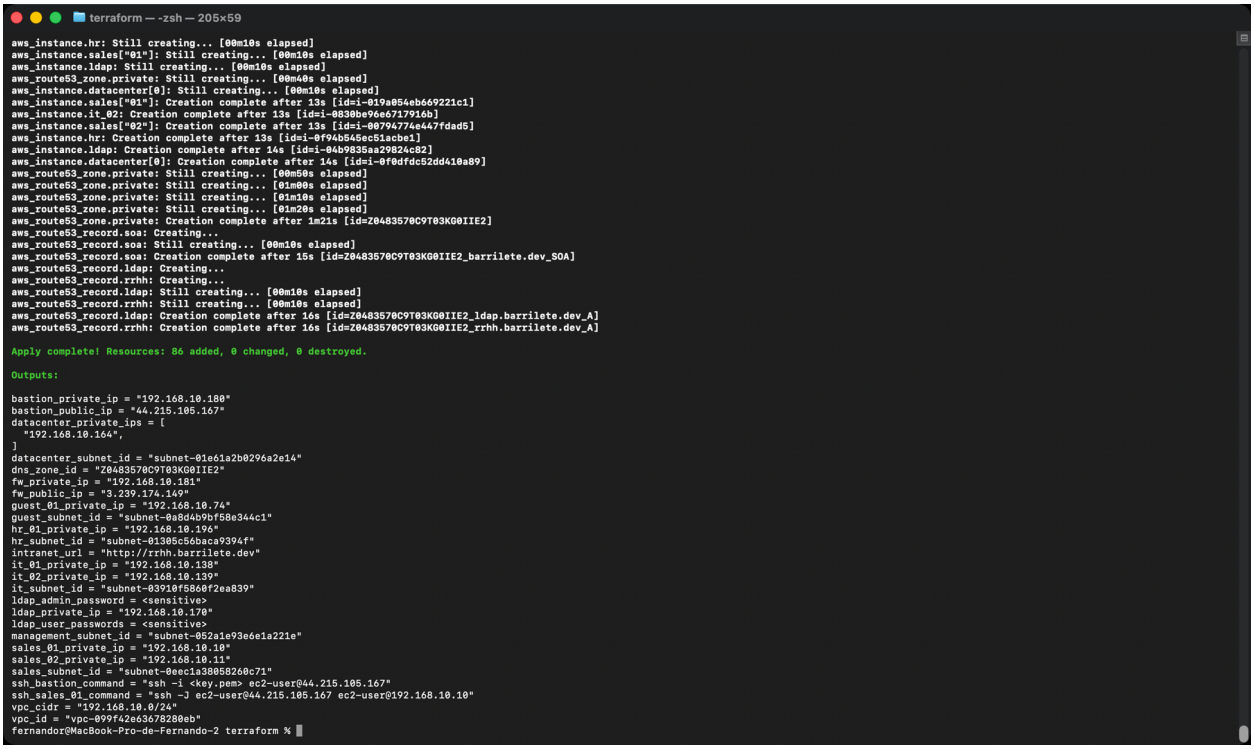


Figura 2. Despliegue de la Fase 2 con terraform apply

Este es el resultado del script de pruebas, que revisa todos los requerimientos de la fase: intranet y DNS, Visitas, TI, LDAP, hardening y firewall. Pasaron las 22 pruebas.

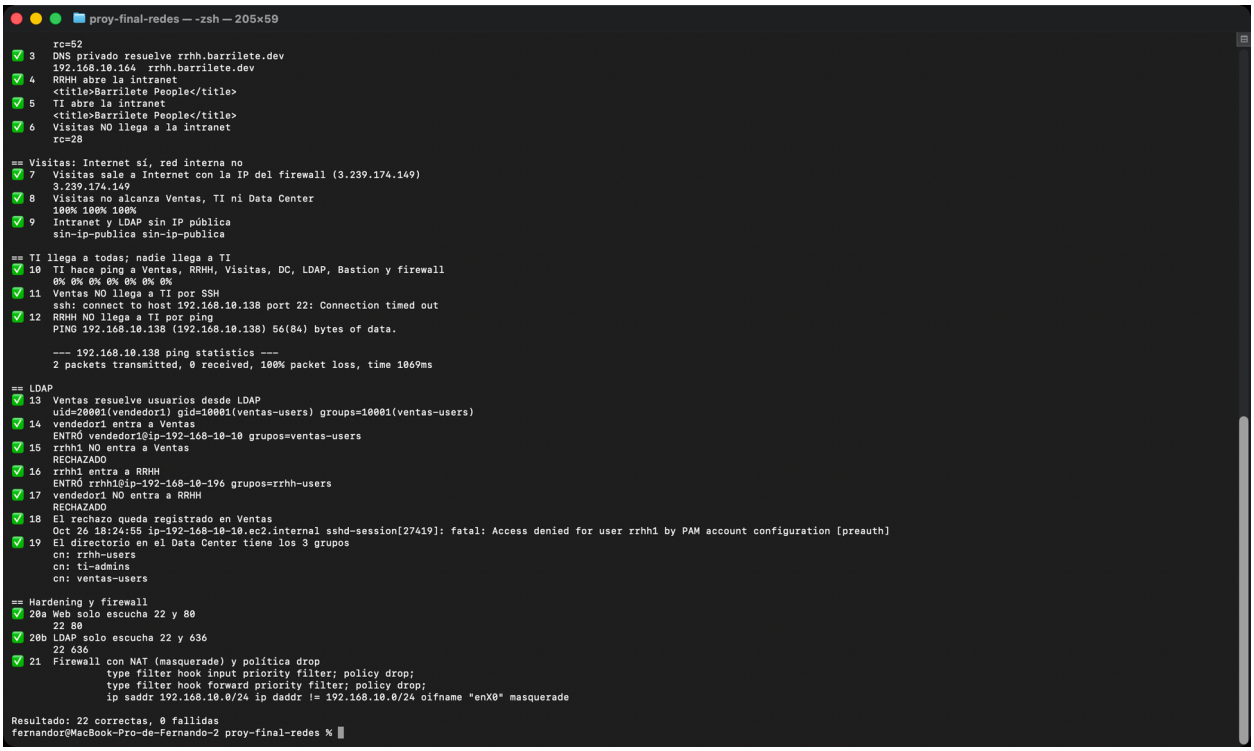


Figura 3. Resultado de smoke-tests.sh: 22 correctas y 0 fallidas

Las reglas de nftables en el firewall. Las cadenas input y forward tienen política drop. En forward solo se permite la salida de la VPC hacia Internet por 80, 443 e ICMP, y la tabla nat hace masquerade del tráfico de 192.168.10.0/24 hacia afuera.

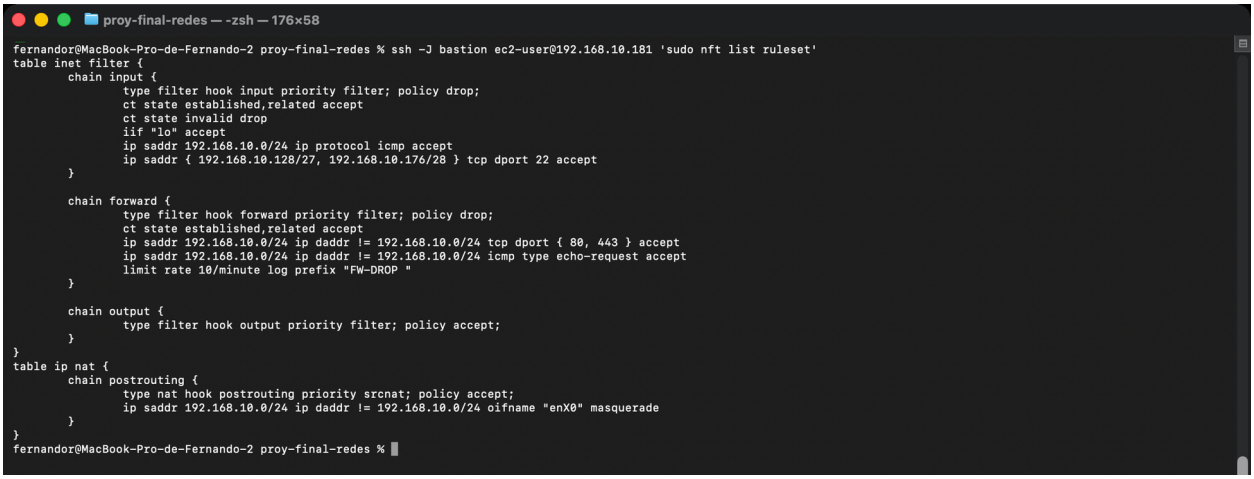


Figura 4. Reglas del firewall (nft list ruleset)

Desde guest-01 consultamos nuestra IP pública y la respuesta es 3.239.174.149, que es la misma IP pública del firewall que muestra Terraform. Esto confirma que Visitas sale a Internet a través del NAT.



Figura 5. Visitas sale a Internet con la IP del firewall

Desde sales-01, HTTPS a github.com responde 200, pero una conexión al puerto 22 del mismo servidor no pasa, porque el firewall solo reenvía 80, 443 y ping.



Figura 6. El firewall deja salir HTTPS y bloquea SSH hacia Internet

Desde guest-01 hicimos ping a Ventas (.10), TI (.138) y Data Center (.164). Los tres pierden el 100 % de los paquetes por la NACL de Visitas.



Figura 7. Visitas no alcanza las demás subredes

Visitas sí resuelve rrhh.barrilete.dev porque está en la misma VPC, pero al intentar abrir la página curl se queda esperando hasta agotar el tiempo (código 28). La intranet no es accesible desde Visitas.



Figura 8. Visitas resuelve el nombre pero no llega a la intranet

Desde la laptop, nslookup no encuentra rrhh.barrilete.dev (NXDOMAIN), porque la zona es privada. Desde sales-01, en cambio, los nombres rrhh y ldap resuelven a sus IP del Data Center.



Figura 9. La zona barrilete.dev solo existe dentro de la VPC

Desde Ventas (.10), RRHH (.196) y TI (.138) pedimos http://rrhh.barrilete.dev y las tres reciben la página Barrilete People.

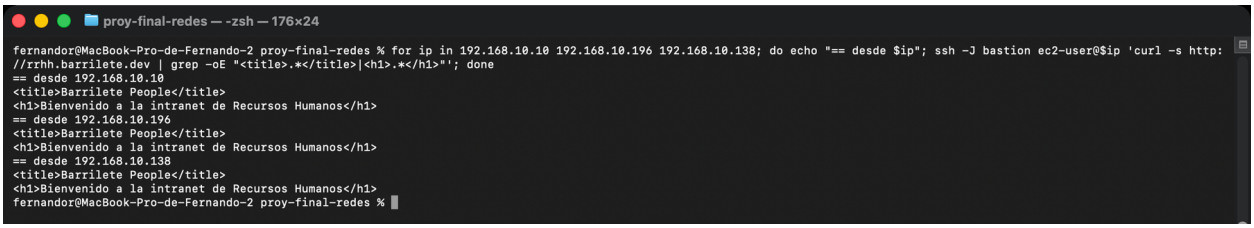


Figura 10. Ventas, RRHH y TI abren la intranet por nombre

Si pedimos la página usando la IP 192.168.10.164 en lugar del nombre, Nginx cierra la conexión sin responder (Empty reply, código 52 de curl). Así la intranet solo funciona por su nombre de dominio.



Figura 11. Por IP la intranet no responde (444)

La intranet abierta en el navegador con http://rrhh.barrilete.dev. Para verla desde la laptop usamos un túnel SOCKS por el Bastion (ssh -D), de modo que la petición sale desde dentro de la VPC.

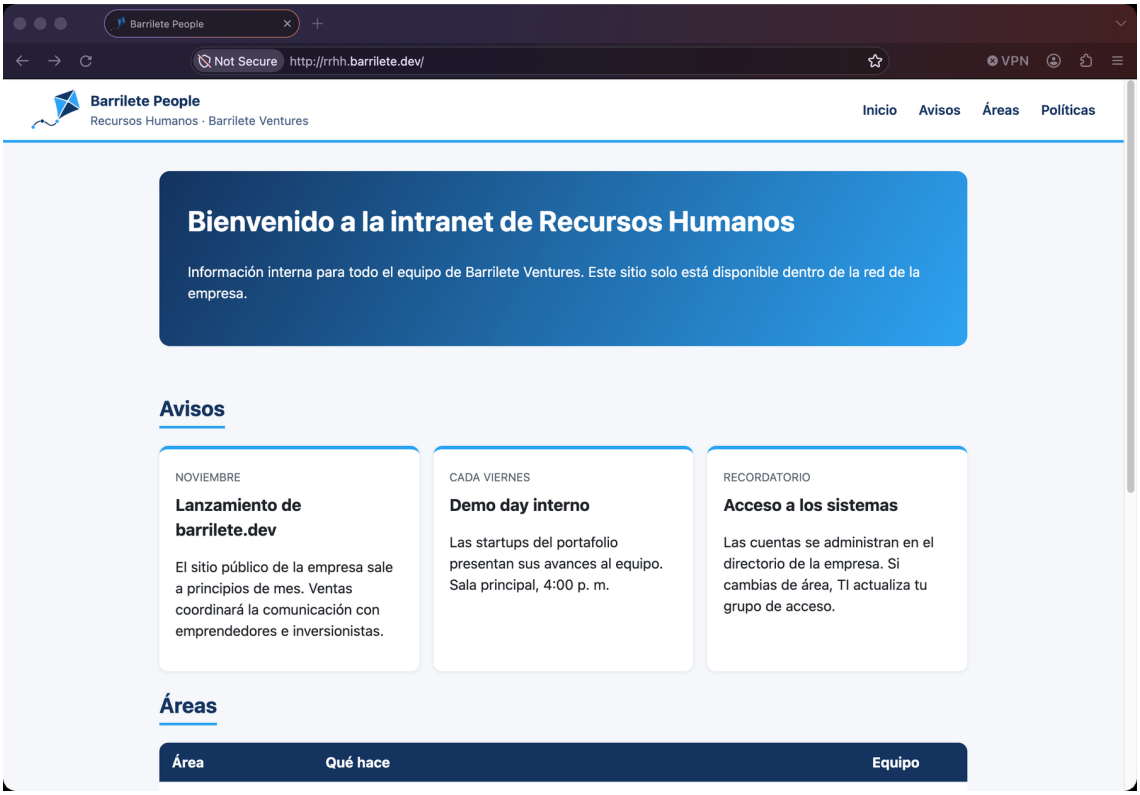


Figura 12. Página Barrilete People en el navegador

Desde it-01 hicimos ping a una instancia de cada subred, al Bastion y al firewall. Todos responden con 0 % de pérdida.



Figura 13. TI llega a todas las subredes

En el sentido contrario, el SSH de sales-01 a it-01 se queda sin respuesta (Connection timed out) y el ping de hr-01 a it-01 pierde el 100 % de los paquetes.

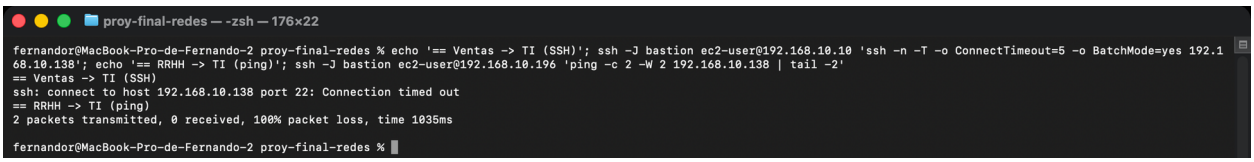


Figura 14. Ventas y RRHH no llegan a TI

Con ldapsearch en dc-ldap se ven los tres grupos con sus miembros y los tres usuarios dentro de dc=barrilete,dc=dev. Al final, id en sales-01 muestra que el sistema ya reconoce a vendedor1 y rrhh1 con los UID y GID del directorio.

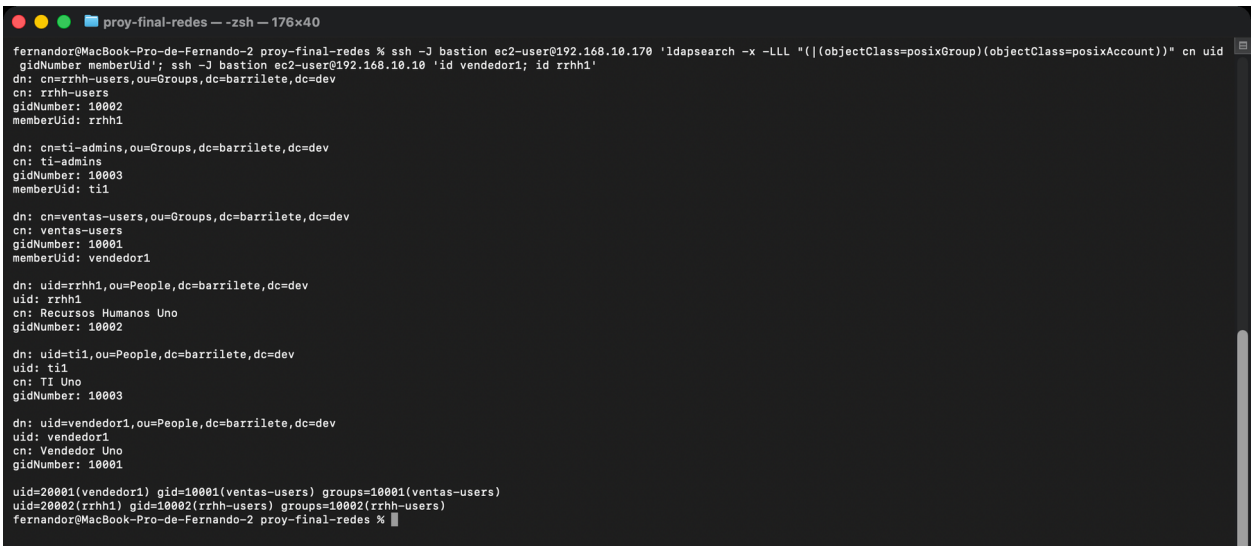


Figura 15. Usuarios y grupos en el servidor LDAP

vendedor1 entra a sales-01 con su contraseña de LDAP. whoami, id y pwd muestran que la sesión es del usuario del directorio, del grupo ventas-users, y que se le creó su directorio personal.



Figura 16. vendedor1 entra a Ventas con su cuenta LDAP

rrhh1 intenta entrar a sales-01 y la conexión se cierra. En el log de sshd de sales-01 se ve por qué: la contraseña es correcta (authentication success), pero SSSD niega el acceso (Access denied for user rrhh1) porque rrhh-users no está autorizado en Ventas.

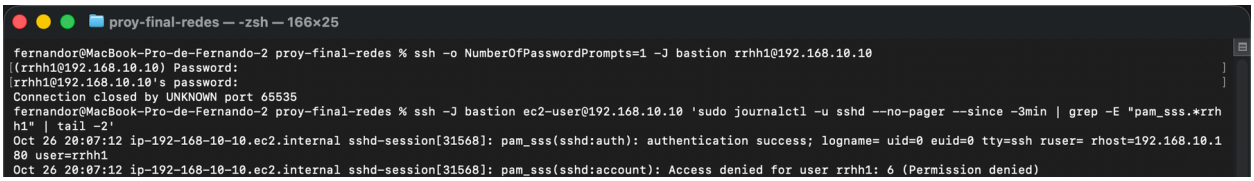


Figura 17. rrhh1 es rechazado en Ventas

En RRHH pasa lo contrario. rrhh1 entra a hr-01 con su grupo rrhh-users, y vendedor1 es rechazado aunque su contraseña sea correcta.

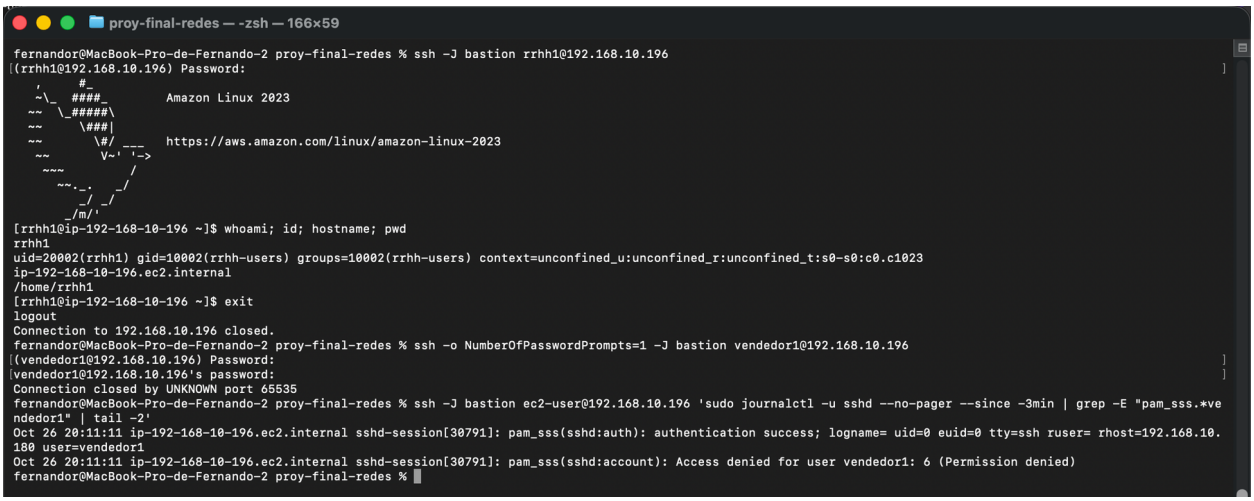


Figura 18. rrhh1 entra a RRHH y vendedor1 es rechazado

El registro de sshd en sales-01 durante la ejecución del script de pruebas. Queda el login aceptado de vendedor1 y los accesos denegados de rrhh1 por la configuración de PAM, que es la que aplica SSSD.

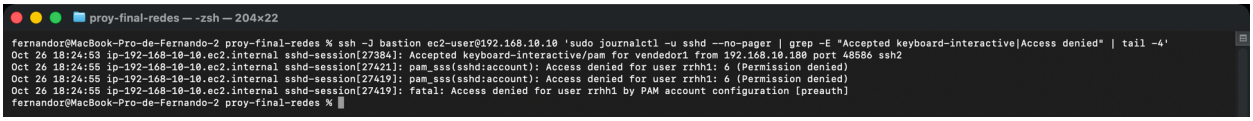


Figura 19. Registro de los accesos en Ventas

Los puertos que escuchan en los servidores. En la web solo están SSH (22) y HTTP (80), en LDAP solo SSH y LDAPS (636), y en el firewall solo SSH.

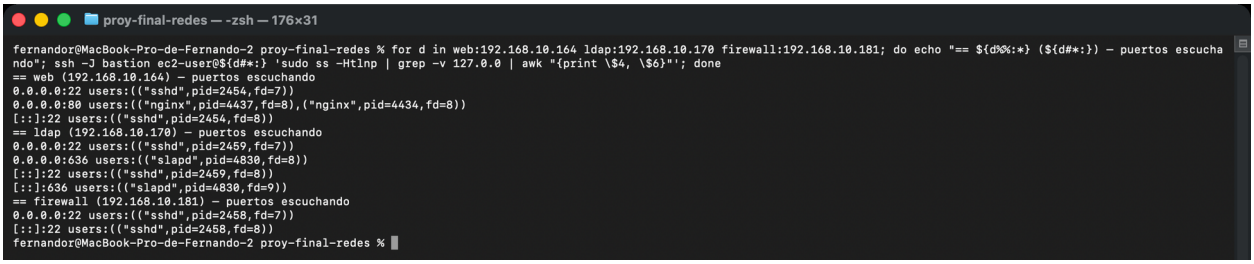


Figura 20. Puertos abiertos en la web, LDAP y el firewall

Documentación consultada

- Amazon Web Services. (s. f.). Work with private hosted zones. Amazon Route 53 Developer Guide. https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/hosted-zones-private.html
- Amazon Web Services. (s. f.). NAT instances. Amazon VPC User Guide. https://docs.aws.amazon.com/vpc/latest/userguide/VPC_NAT_Instance.html
- Amazon Web Services. (s. f.). Control subnet traffic with network access control lists. Amazon VPC User Guide. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html
- Amazon Web Services. (s. f.). Control traffic to your AWS resources using security groups. Amazon VPC User Guide. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html
- HashiCorp. (s. f.). AWS Provider y TLS Provider. Terraform Registry. https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- Netfilter Project. (s. f.). Performing Network Address Translation (NAT). nftables wiki. https://wiki.nftables.org/wiki-nftables/index.php/Performing_Network_Address_Translation_(NAT)
- NGINX. (s. f.). How nginx processes a request. https://nginx.org/en/docs/http/request_processing.html
- OpenLDAP Foundation. (s. f.). OpenLDAP Software 2.4 Administrator's Guide. https://www.openldap.org/doc/admin24/

- SSSD Project. (s. f.). sssd-ldap y sssd-simple. https://sssd.io/docs/
- Sermersheim, J. (2006). Lightweight Directory Access Protocol (LDAP): The Protocol (RFC 4511). IETF. https://www.rfc-editor.org/rfc/rfc4511